

Кортежи Списки И другие неприятности

Кортеж

```
("Haskell", 2010)
```

```
(1, True, "Third", 4.0, '5')
```

```
ghci> (1, True)
(1,True)
ghci> :t (1, True)
(1, True) :: Num a => (a,
Bool)
```

Кортеж

```
("Haskell", 2010)
```

```
(1, True, "Third", 4.0, '5')
```

```
(,) "Haskell" 2010
```

```
(,,,,) 1 True "Third" 4.0 '5'
```

```
ghci> :t (,)
(,) :: a → b → (a, b)
```

```
ghci> :t (,,,,)
(,,,,) :: a → b → c → d → e → (a, b, c, d, e)
```

Функции работы с кортежами

```
ghci> fst (1, True)
1
ghci> snd (1, True)
True
```

Функции работы с кортежами. Каррирование

```
ghci> :t curry
curry :: ((a, b) → c) → a → b → c

ghci> curry fst 1 True
1
```

Функции работы с кортежами. Декаррирование

```
ghci> :t uncurry
uncurry :: (a → b → c) → (a, b) → c

ghci> uncurry div (3,2)
1
```

Функции работы с кортежами

```
hasAccess :: String → String → Bool
hasAccess role page
  | role == "admin" = True
  | role == "user" && page == "main" = True
  | otherwise = False
```

Функции работы с кортежами

```
hasAccess :: String → String → Bool
hasAccess role page
    | role == "admin" = True
    | role == "user" && page == "main" = True
    | otherwise = False
```

```
request = ("admin", "dashboard")
```

```
checkRequest :: (String, String) → Bool
checkRequest request = ??
```

Функции работы с кортежами. Пример 1

```
hasAccess :: String → String → Bool
hasAccess role page
  | role == "admin" = True
  | role == "user" && page == "main" = True
  | otherwise = False
```

```
request = ("admin", "dashboard")
-- v1
checkRequest :: (String, String) → Bool
checkRequest request = hasAccess (fst request) (snd request)
```

Функции работы с кортежами. Пример 2

```
hasAccess :: String → String → Bool
hasAccess role page
  | role == "admin" = True
  | role == "user" && page == "main" = True
  | otherwise = False
```

```
request = ("admin", "dashboard")
-- v2
checkRequest :: (String, String) → Bool
checkRequest request = uncurry hasAccess request
```

Функции работы с кортежами. Пример 2.5

```
hasAccess :: String → String → Bool
hasAccess role page
  | role == "admin" = True
  | role == "user" && page == "main" = True
  | otherwise = False
```

```
request = ("admin", "dashboard")
-- v2.5
checkRequest :: (String, String) → Bool
checkRequest = uncurry hasAccess
```

Функции работы с кортежами. Пример 3

```
hasAccess :: String → String → Bool
hasAccess role page
  | role == "admin" = True
  | role == "user" && page == "main" = True
  | otherwise = False
```

```
request = ("admin", "dashboard")
-- v3
checkRequest :: (String, String) → Bool
checkRequest (role, page) = hasAccess role page
```

Списки

```
myList = [1, 2, 3, 4]
```

```
[Int]      -- Список целых чисел  
[Char]     -- Список символов, aka String  
[String]   -- Список строк
```

: и ++

: - добавление элемента
в голову списка

```
ghci> :i (:)
type List :: * -> *
data List a = ... | a : [a]
               -- Defined in `GHC.Types'
infixr 5 :
```

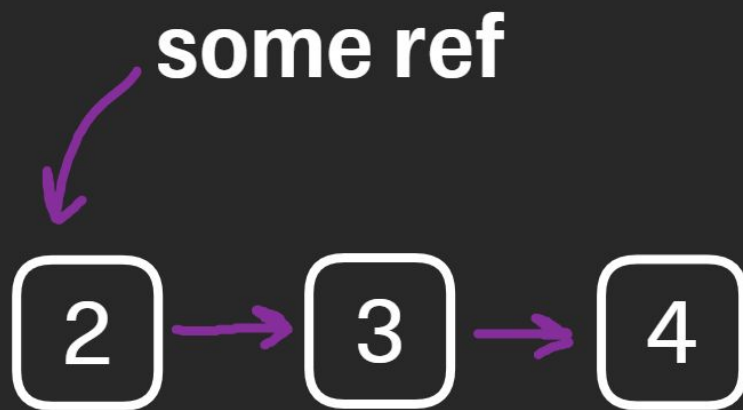
++ - конкатенация двух
списков

```
ghci> :i (++)
(++) :: [a] -> [a] -> [a]
-- Defined in
`GHC.Internal.Base'
infixr 5 ++
```

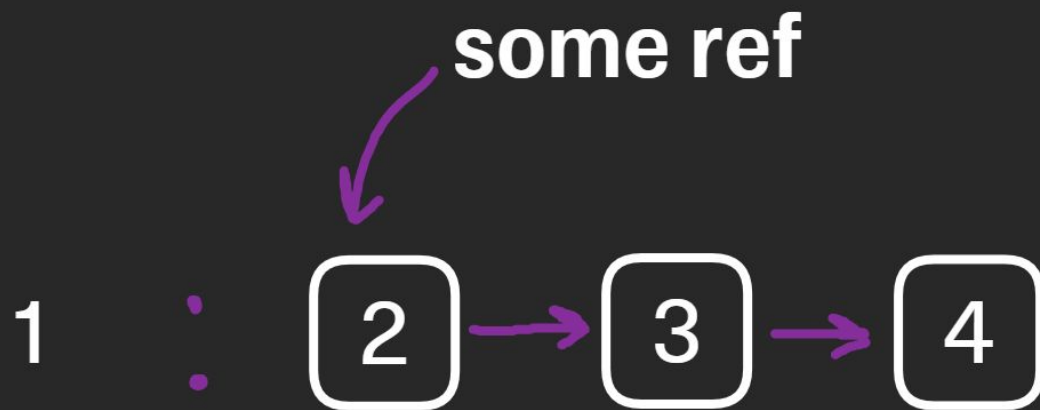
```
ghci> 1 : 2 : 3 : 4 : []
[1,2,3,4]
```

```
ghci> [1] ++ [2] ++ [3, 4]
[1,2,3,4]
```

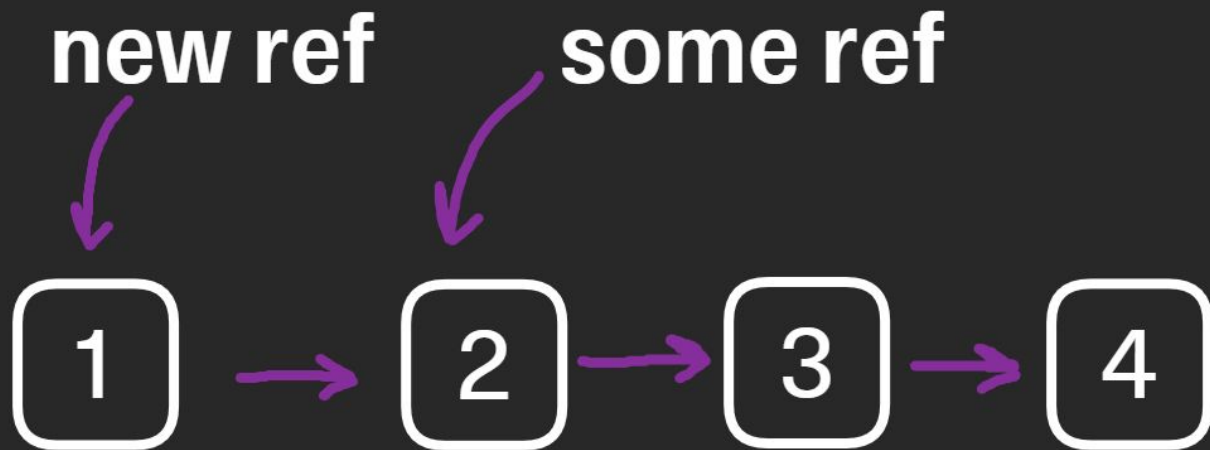
: - друг



: - друг

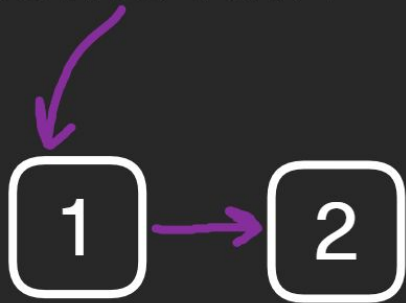


: - друг



++ - не друг

some ref1



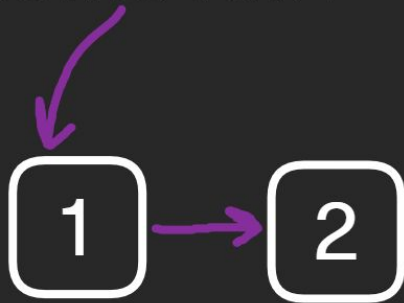
++

some ref2



++ - не друг

some ref1



some ref2

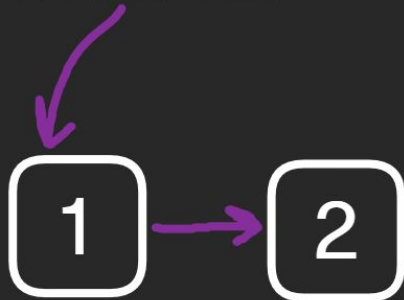


++

ok, lets copy!

++ - не друг

some ref1



some ref2

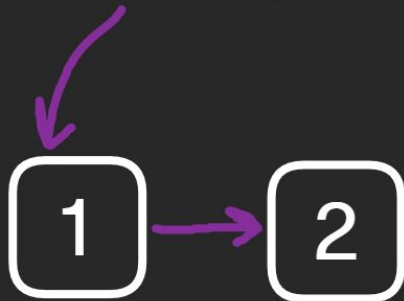


absolutely new ref



++ - не друг

some ref1



some ref2



absolutely new ref

Выводы:

- Алгоритмическая сложность $O(n)$, где n - длина первого списка
- Иногда проще `reverse` и :

Стандартные функции работы с массивами

```
head [10, 20, 30]      -- 10
tail [10, 20, 30]      -- [20, 30]
[10, 20, 30] !! 1      -- 20
null []                -- True
```

Стандартные функции работы с массивами

```
head [10, 20, 30]      -- 10
tail [10, 20, 30]      -- [20, 30]
[10, 20, 30] !! 1      -- 20
null []                -- True
```

Но есть одно но

```
head []                -- exception
tail []                -- exception
[10, 20, 30] !! 10     -- exception
```

Стандартные функции работы с массивами

```
head [10, 20, 30]      -- 10
tail [10, 20, 30]      -- [20, 30]
[10, 20, 30] !! 1      -- 20
null []                -- True
```

Но есть одно но

```
head []                -- exception
tail []                -- exception
[10, 20, 30] !! 10     -- exception
```

Совет от компилятора:

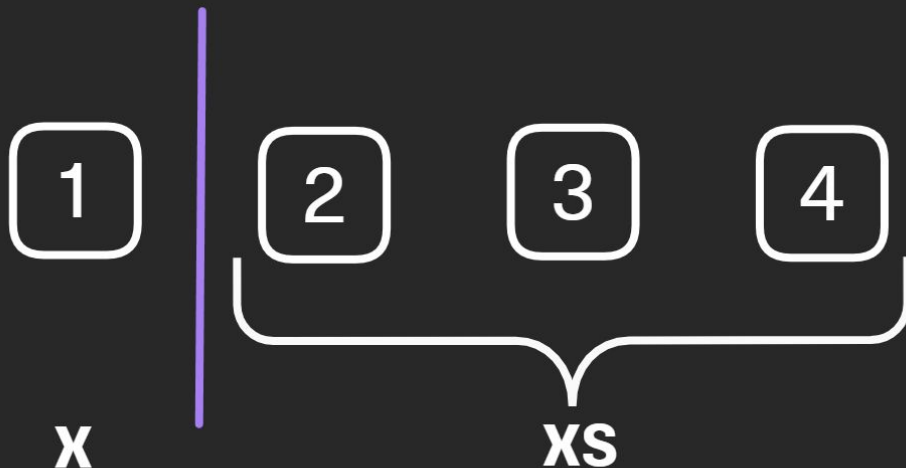
Старайтесь не использовать частичные функции

Pattern matching для массивов

```
sum' :: [Int] → Int  
sum' []      = 0  
sum' (x:xs)  = x + sum' xs
```

Pattern matching для массивов

```
sum' :: [Int] → Int  
sum' []      = 0  
sum' (x:xs) = x + sum' xs
```



Pattern matching для массивов

```
sum' :: [Int] → Int
sum' []      = 0
sum' (x1:x2:xs) = x1 + x2 + sum' xs
-- warning: Patterns of type
-- '[Int]' not matched
```

Pattern matching для массивов. Map

```
map :: (a → b) → [a] → [b]
map _ [] = []
map f (x:xs) = f x : map f xs
```

Pattern matching для массивов. Map

```
map :: (a → b) → [a] → [b]
map _ [] = []
map f (x:xs) = f x : map f xs
```

```
ghci> import Data.Char
ghci> map toUpper "privet"
"PRIVET"
```

Pattern matching для массивов. Filter

```
filter :: (a → Bool) → [a] → [a]
filter _ [] = []
filter p (x:xs)
    | p x = x : filter p xs
    | otherwise = filter p xs
```

Pattern matching для массивов. Filter

```
filter :: (a → Bool) → [a] → [a]
filter _ [] = []
filter p (x:xs)
    | p x = x : filter p xs
    | otherwise = filter p xs
```

```
ghci> filter (\x → x `mod` 2 == 0) [1, 2, 3, 10, 11]
[2,10]
```

take, drop, any, all

```
ghci> let arr = [42, 1, 4, 3, 42]
```

```
ghci> take 2 arr  
[42,1]
```

```
ghci> drop 2 arr  
[4,3,42]
```

```
ghci> all (> 5) arr  
False
```

```
ghci> any (> 5) arr  
True
```

```
ghci> any (> 5) [6, undefined]  
True
```

```
ghci> all (> 5) [6, undefined]  
*** Exception: Prelude.undefined
```

Свёртки

```
sum' :: [Int] → Int
sum' []      = 0          -- initial accumulator value
sum' (x:xs) = x + sum' xs -- operation on head; then recursion
```

Свёртки

```
sum' :: [Int] → Int
sum' []      = 0          -- initial accumulator value
sum' (x:xs) = x + sum' xs -- operation on head; then recursion
```

```
fun :: [a] → b
fun [] = initialValue
fun (x:xs) = x `op` fun xs
```

Свёртки

```
sum' :: [Int] → Int
sum' []      = 0          -- initial accumulator value
sum' (x:xs) = x + sum' xs -- operation on head; then recursion
```

```
fun :: [a] → b
fun [] = initialValue
fun (x:xs) = x `op` fun xs
```

```
fun :: (a → b → b) → b → [a] → b
```

Свёртки

```
sum' :: [Int] → Int
sum' []      = 0          -- initial accumulator value
sum' (x:xs) = x + sum' xs -- operation on head; then recursion
```

```
fun :: [a] → b
fun [] = initialValue
fun (x:xs) = x `op` fun xs
```

```
fun :: (a → b → b) → b → [a] → b
fun op accumulator [] = accumulator
```

Свёртки

```
sum' :: [Int] → Int
sum' []      = 0          -- initial accumulator value
sum' (x:xs) = x + sum' xs -- operation on head; then recursion
```

```
fun :: [a] → b
fun [] = initialValue
fun (x:xs) = x `op` fun xs
```

```
fun :: (a → b → b) → b → [a] → b
fun op accumulator [] = accumulator
fun op accumulator (x:xs) = x `op` (fun op accumulator xs)
```

Свёртки. Foldr

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr op accumulator [] = accumulator
foldr op accumulator (x:xs) = x `op` (foldr op accumulator xs)
```

$$\begin{aligned} & \text{foldr } (\otimes) v [e_0, e_1, e_2, \dots, e_{n-1}, e_n] \\ &= \\ & e_0 \otimes (e_1 \otimes (e_2 \otimes \dots \otimes (e_{n-1} \otimes (e_n \otimes v)) \dots)) \end{aligned}$$

Свёртки. Foldr

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr op accumulator [] = accumulator
foldr op accumulator (x:xs) = x `op` (foldr op accumulator xs)
```

$$\begin{aligned} & \text{foldr } (\otimes) v [e_0, e_1, e_2, \dots, e_{n-1}, e_n] \\ &= \\ & e_0 \otimes (e_1 \otimes (e_2 \otimes \dots \otimes (e_{n-1} \otimes (e_n \otimes v)) \dots)) \end{aligned}$$

```
foldr (+) 0 [1, 2, 3]                -- 1 + (2 + (3 + 0)) = 6
foldr (:) [0] [1, 2, 3, 4]           -- 1 : (2 : (3 : (4 : [0]))) = [1,2,3,4,0]
foldr (\x accum -> [x] : accum) [] [1, 2] -- [1] : ([2] : []) = [[1],[2]]
```

Свёртки. Foldl

```
foldl :: (b -> a -> b) -> b -> [a] -> b  
foldl op accumulator [] = accumulator  
foldl op accumulator (x:xs) = foldl op (accumulator `op` x) xs
```

$$\begin{aligned} & \text{foldl } (\otimes) v [e_0, e_1, e_2, \dots, e_{n-1}, e_n] \\ &= \\ & (((v \otimes e_0) \otimes e_1) \otimes e_2) \otimes \dots \otimes e_{n-1}) \otimes e_n \end{aligned}$$

```
foldl (+) 0 [1, 2, 3]    -- ((0 + 1) + 2) + 3 = 6  
foldl (\acc x -> x : acc) [] [1, 2, 3] -- (3 : (2 : (1 : []))) = [3, 2, 1]
```

Performance. `Foldr`, `foldl`, `foldl'`

```
ghci> let xs = [1 .. 200000]
```

```
ghci> :set +s
```

```
ghci> foldr (+) 0 xs
```

```
20000100000
```

```
(0.02 secs, 32,347,032 bytes)
```

```
ghci> foldl (+) 0 xs
```

```
20000100000
```

```
(0.05 secs, 32,275,800 bytes)
```

```
ghci> foldl' (+) 0 xs
```

```
20000100000
```

```
(0.01 secs, 17,656,672 bytes)
```

$$\begin{aligned} \text{foldr } (\otimes) v [e_0, e_1, e_2, \dots, e_{n-1}, e_n] \\ = \\ e_0 \otimes (e_1 \otimes (e_2 \otimes \dots \otimes (e_{n-1} \otimes (e_n \otimes v)) \dots)) \end{aligned}$$

```
ghci> foldr (&&) True (False : undefined)
-- = False && foldr (&&) True undefined
False
```

```
ghci> foldr (\x acc → x == 5 || acc) False [1 ..]
True
```

$$\begin{aligned} \text{foldl } (\otimes) v [e_0, e_1, e_2, \dots, e_{n-1}, e_n] \\ = \\ (((v \otimes e_0) \otimes e_1) \otimes e_2) \otimes \dots \otimes e_{n-1} \otimes e_n \end{aligned}$$

```
ghci> foldl (&&) True (False : undefined)
-- = foldl (&&) (True && False) undefined
*** Exception: Prelude.undefined
```

```
ghci> foldl (\acc x → acc || x == 5) False [1 ..]
-- never ends
```

<https://blog.haskell.org/foldl-and-foldr/>

Списки и генераторы

-- конечные

[1 .. 5] -- [1, 2, 3, 4, 5]

[1, 3 .. 10] -- [1, 3, 5, 7, 9]

[10, 9 .. 7] -- [10, 9, 8, 7]

-- бесконечные

[1 ..] -- [1, 2, 3, 4, 5, 6, 7..]

[2, 4 ..] -- [2, 4, 6, 8, 10, 12, 14..]

[10, 9 ..] -- [10, 9, 8, 7, 6, 5, 4, 3 ..]

-- бесконечные, но имеющие смысл

take 5 [1 ..] -- [1, 2, 3, 4, 5]

take 5 [2, 4 ..] -- [2, 4, 6, 8, 10]

take 5 [10, 9 ..] -- [10, 9, 8, 7, 6]

Списки и генераторы

```
ghci> [n * n | n <- [1..10]]  
[1,4,9,16,25,36,49,64,81,100]
```

Списки и генераторы

```
ghci> [n * n | n <- [1..10]]  
[1,4,9,16,25,36,49,64,81,100]
```

```
ghci> take 6 [n | n <- [1 ..], even n]  
[2,4,6,8,10,12]
```

Списки и генераторы

```
ghci> [n * n | n <- [1..10]]  
[1,4,9,16,25,36,49,64,81,100]
```

```
ghci> take 6 [n | n <- [1 ..], even n]  
[2,4,6,8,10,12]
```

```
ghci> take 6 [(n, n * n) | n <- [1 ..], even n]  
[(2,4),(4,16),(6,36),(8,64),(10,100),(12,144)]
```

Списки и генераторы

```
ghci> [n * n | n <- [1..10]]  
[1,4,9,16,25,36,49,64,81,100]
```

```
ghci> take 6 [n | n <- [1 ..], even n]  
[2,4,6,8,10,12]
```

```
ghci> take 6 [(n, n * n) | n <- [1 ..], even n]  
[(2,4),(4,16),(6,36),(8,64),(10,100),(12,144)]
```

```
ghci> [(a, b) | a <- [1 .. 3], b <- [4 ..5]]  
[(1,4),(1,5),(2,4),(2,5),(3,4),(3,5)]
```



ВСЁ!